

# Software Architecture and Design Specification

## Project: Vehicle Parking System

**Version:** 1.0

**Date:** 06-09-2026

**Status:** Draft

### Revision History

| Version | Date       | Author       | Change Summary                                           |
|---------|------------|--------------|----------------------------------------------------------|
| 1.0     | 06-09-2026 | Thrupthi G D | Initial SAD draft — architecture, components, tech stack |

### Table of Contents

1. Introduction
  2. Document Overview
  3. Architecture
  4. Design (*to be completed*)
- 

## 1. Introduction

### 1.1 Purpose

This document specifies the software architecture and design of the Vehicle Parking System (VPS). It provides a structural blueprint of the system components, their interactions, technology choices, and security design for use by developers, QA engineers, and the course assessor.

### 1.2 Scope

Covers the architecture of all VPS software components: the customer entry/exit terminal, slot management engine, fee calculation service, payment integration, admin portal, and the underlying database and API layer. Hardware firmware and third-party payment gateway internals are excluded.

### 1.3 Audience

Developers, QA Engineers, Security Auditors, Course Instructors, and Maintenance Teams.

## 1.4 Definitions

| Term    | Definition                                   |
|---------|----------------------------------------------|
| VPS     | Vehicle Parking System                       |
| ADR     | Architectural Decision Record                |
| PCI-DSS | Payment Card Industry Data Security Standard |
| TLS     | Transport Layer Security                     |
| RBAC    | Role-Based Access Control                    |
| ALPR    | Automatic License Plate Recognition          |
| API     | Application Programming Interface            |

---

## 2. Document Overview

### 2.1 How to Use This Document

This document provides architectural deliverables including component descriptions, UML diagrams, architectural decision rationale, threat model, and API design. Read Section 3 for the big-picture structure, then Section 4 for detailed design of individual components.

### 2.2 Related Documents

| Document                                  | Location                                |
|-------------------------------------------|-----------------------------------------|
| SRS — Software Requirements Specification | docs/SRS/SRS_Vehicle_Parking_System.pdf |
| STP — Software Test Plan                  | docs/STP/STP_VehicleParkingSystem.md    |

---

## 3. Architecture

### 3.1 Goals & Constraints

**Goals:** - Secure handling of vehicle and payment data - Real-time slot tracking with < 5 second response time - 99.9% monthly availability - Modular design for easy maintenance and future extension

**Constraints:** - PCI-DSS compliance for all payment flows - ALPR camera integration is vendor-specific (must be abstracted) - System must support degraded offline mode during network outages - Hardware barrier APIs vary by vendor and must be wrapped behind an interface layer

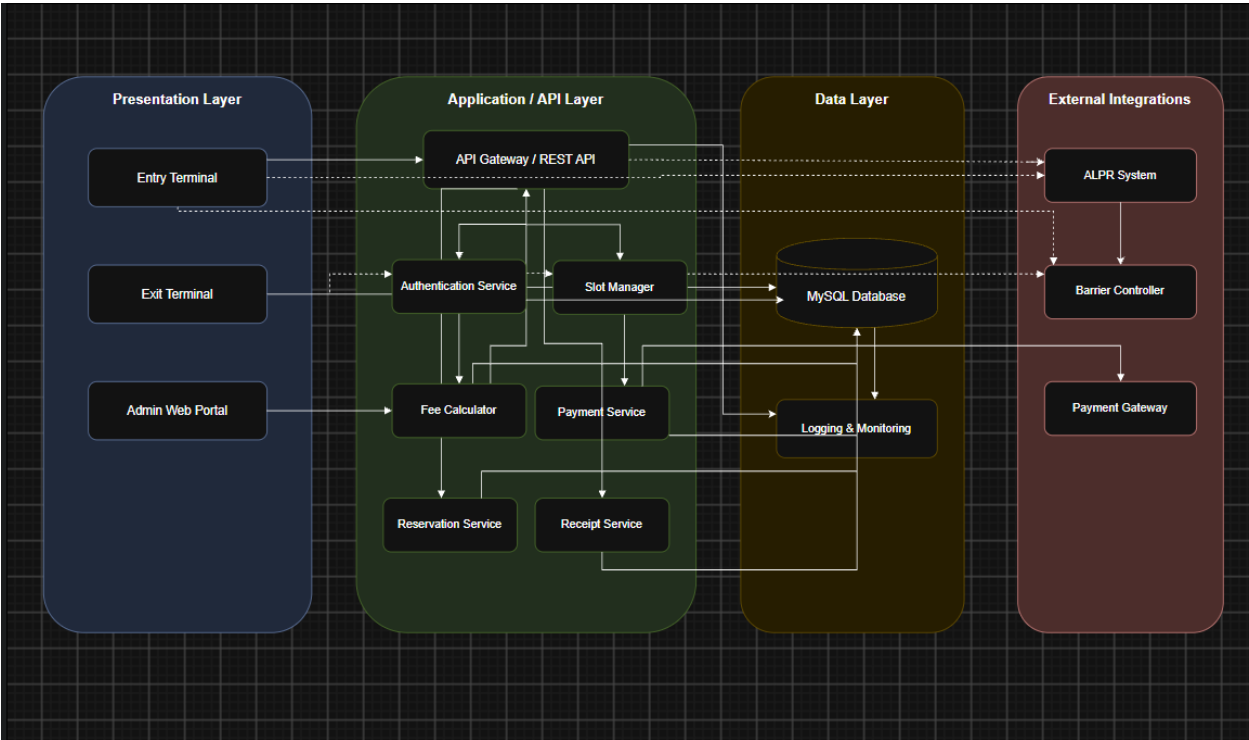
---

### 3.2 Stakeholders & Concerns

| Stakeholder         | Primary Concerns                            |
|---------------------|---------------------------------------------|
| Customers / Drivers | Fast entry/exit, accurate billing, clear UI |

| Stakeholder             | Primary Concerns                                  |
|-------------------------|---------------------------------------------------|
| Parking Operators       | Real-time occupancy, revenue reports, reliability |
| Bank / Payment Provider | PCI-DSS compliance, secure data handling          |
| Regulators              | Audit logs, data retention, accessibility         |
| Developers              | Modularity, clear API contracts, testability      |
| Maintenance Technicians | Diagnostic access, error logging                  |

3.3 Component (UML) Diagram



3.4 Component Descriptions

| Component                | Responsibility                                                                              |
|--------------------------|---------------------------------------------------------------------------------------------|
| <b>Entry Terminal UI</b> | Captures license plate (manual or ALPR), displays slot info, prints/shows ticket            |
| <b>Exit Terminal UI</b>  | Scans ticket, displays fee, handles payment, prints receipt, triggers barrier               |
| <b>Admin Web Portal</b>  | Occupancy dashboard, rate configuration, report generation, user management                 |
| <b>API Gateway</b>       | Single entry point for all client requests; handles routing, rate limiting, TLS termination |

| Component                         | Responsibility                                                                                |
|-----------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Auth Service</b>               | Validates credentials, issues session tokens, enforces RBAC for all roles                     |
| <b>Slot Manager</b>               | Tracks real-time slot occupancy, handles assignment, categorisation, and availability updates |
| <b>Fee Calculator</b>             | Computes parking charges based on duration and configured rate schedule                       |
| <b>Payment Service</b>            | Processes payments via gateway, confirms transactions, triggers receipt generation            |
| <b>Receipt Service</b>            | Generates and delivers receipts (print or digital)                                            |
| <b>Reservation Service</b>        | Manages advance bookings and automatic grace-period release                                   |
| <b>Logging &amp; Monitoring</b>   | Captures audit events, error logs, uptime metrics; ships to monitoring backend                |
| <b>Database (MySQL)</b>           | Stores vehicle records, slot states, transactions, rate schedules, user accounts              |
| <b>ALPR / Barrier Adapter</b>     | Abstracts vendor-specific camera and barrier hardware behind a uniform interface              |
| <b>Payment Gateway (External)</b> | Third-party service for card/digital payment authorisation and settlement                     |

### 3.5 Chosen Architecture Pattern and Rationale

#### Pattern chosen: Layered Architecture (with service decomposition)

The system is structured into four layers: 1. **Presentation Layer** — Terminal UIs and Admin Web Portal 2. **Application / API Layer** — API Gateway and business services 3. **Domain / Business Logic Layer** — Slot Manager, Fee Calculator, Reservation Service 4. **Data Layer** — MySQL database and external integrations

**Rationale:** - Layered architecture enforces clear separation of concerns, making each layer independently testable. - Microservices were considered but rejected as overly complex for the scale and team size of this project. - The service decomposition within the application layer (Auth, Slot, Fee, Payment as separate services) allows parallel development by team members without tight coupling.

### 3.6 Technology Stack & Data Stores

| Layer                  | Technology                                                  |
|------------------------|-------------------------------------------------------------|
| Frontend (Terminal UI) | React.js (touch-optimised kiosk mode)                       |
| Admin Portal           | React.js + Tailwind CSS                                     |
| Backend API            | Java, Spring Boot (REST)                                    |
| Authentication         | JWT tokens, Spring Security                                 |
| Database               | MySQL 8.x                                                   |
| Encryption in Transit  | TLS 1.2+ (mandatory), mutual TLS for payment gateway        |
| Encryption at Rest     | AES-256 for sensitive fields (LP data, PAN)                 |
| Hardware Abstraction   | Java adapter layer over vendor serial/network APIs          |
| Monitoring & Logging   | ELK Stack (Elasticsearch, Logstash, Kibana)                 |
| Deployment             | Docker containers, hosted on cloud VM or on-premises server |

### 3.7 Risks & Mitigations

| Risk                             | Impact                  | Mitigation                                             |
|----------------------------------|-------------------------|--------------------------------------------------------|
| ATM/barrier network failure      | Entry/exit blocked      | Offline queuing mode; local cache for slot state       |
| ALPR camera misread              | Wrong vehicle logged    | Manual override at terminal; attendant correction flow |
| Payment gateway downtime         | Customers cannot pay    | Cash fallback mode; attendant manual receipt           |
| Database single point of failure | System unavailable      | DB replication with automated failover                 |
| PCI-DSS non-compliance           | Legal/financial penalty | No plaintext PAN storage; TLS everywhere; annual audit |

### 3.8 Traceability to Requirements

| Requirement ID | Requirement Short     | Component                            |
|----------------|-----------------------|--------------------------------------|
| VPS-F-001      | Capture license plate | ALPR/Barrier Adapter, Entry Terminal |
| VPS-F-002      | Assign slot on entry  | Slot Manager                         |
| VPS-F-003      | Generate entry ticket | Ticket/Receipt Service               |

| Requirement ID | Requirement Short         | Component                   |
|----------------|---------------------------|-----------------------------|
| VPS-F-007      | Calculate parking fee     | Fee Calculator              |
| VPS-F-008      | Accept payment at exit    | Payment Service             |
| VPS-F-013      | Admin occupancy dashboard | Admin Web Portal            |
| VPS-NF-001     | Response time $\leq 5s$   | API Gateway, all services   |
| VPS-NF-003     | PCI-DSS compliance        | Payment Service, Data Layer |
| VPS-SR-001     | TLS 1.2+                  | API Gateway, Network Layer  |
| VPS-SR-003     | RBAC                      | Auth Service                |

### 3.9 Security Architecture

#### Threat Model (STRIDE):

| Threat                        | Example                        | Mitigation                                                                        |
|-------------------------------|--------------------------------|-----------------------------------------------------------------------------------|
| <b>Spoofing</b>               | Fake admin login               | JWT authentication + RBAC; account lockout after 5 failures (VPS-SR-004)          |
| <b>Tampering</b>              | Modifying transaction records  | DB integrity constraints; audit log with tamper-evident hashing                   |
| <b>Repudiation</b>            | Denying a transaction occurred | Timestamped, signed audit logs retained for 5 years (VPS-NF-004)                  |
| <b>Information Disclosure</b> | Payment data exposed in logs   | No PAN/PIN in logs; AES-256 at rest; TLS in transit (VPS-SR-001, VPS-SR-005)      |
| <b>Denial of Service</b>      | Flooding the API               | Rate limiting at API Gateway; load balancing                                      |
| <b>Elevation of Privilege</b> | Customer accessing admin panel | RBAC strictly enforced; roles validated server-side on every request (VPS-SR-003) |

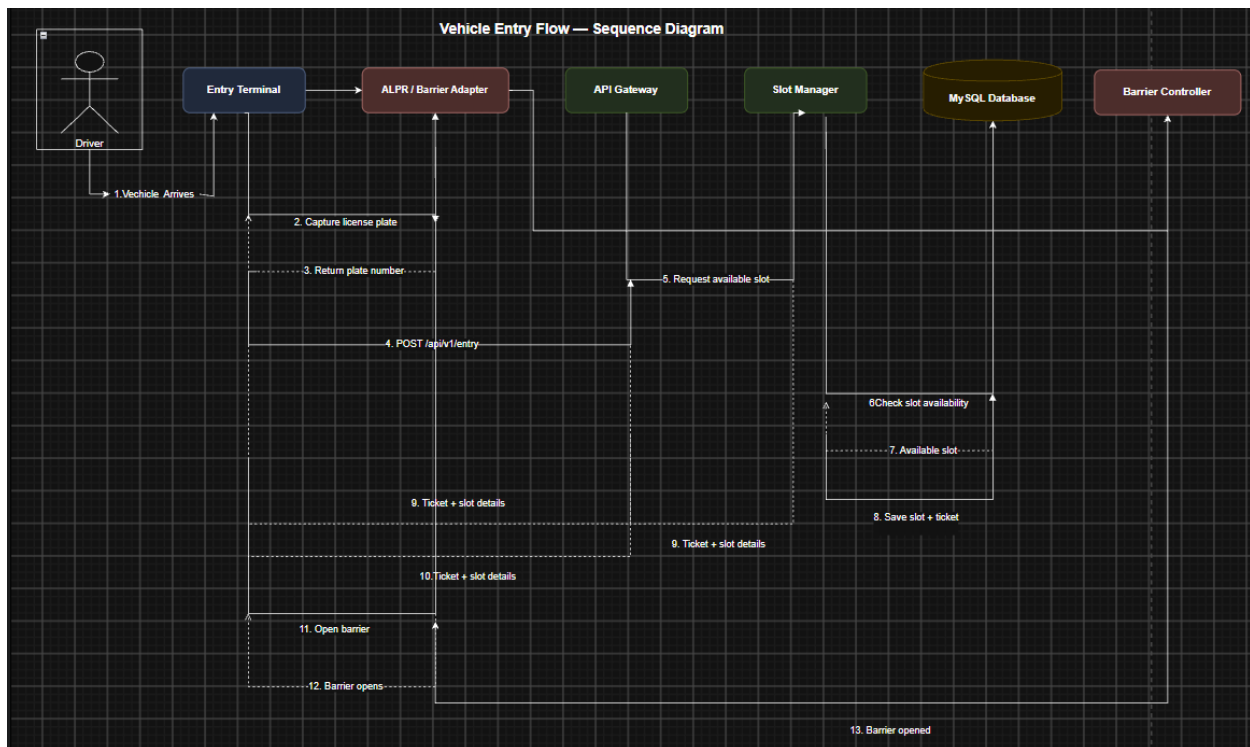
## 4. Design

### 4.1 Design Overview

The VPS follows a request-response model for all terminal interactions. Each transaction (entry, exit, payment) is atomic — if any step fails, the system rolls back and notifies the user. All state changes (slot occupancy, payment status) are persisted to the database before hardware actions (barrier open) are triggered, ensuring consistency even during partial failures.

## 4.2 UML Sequence Diagrams

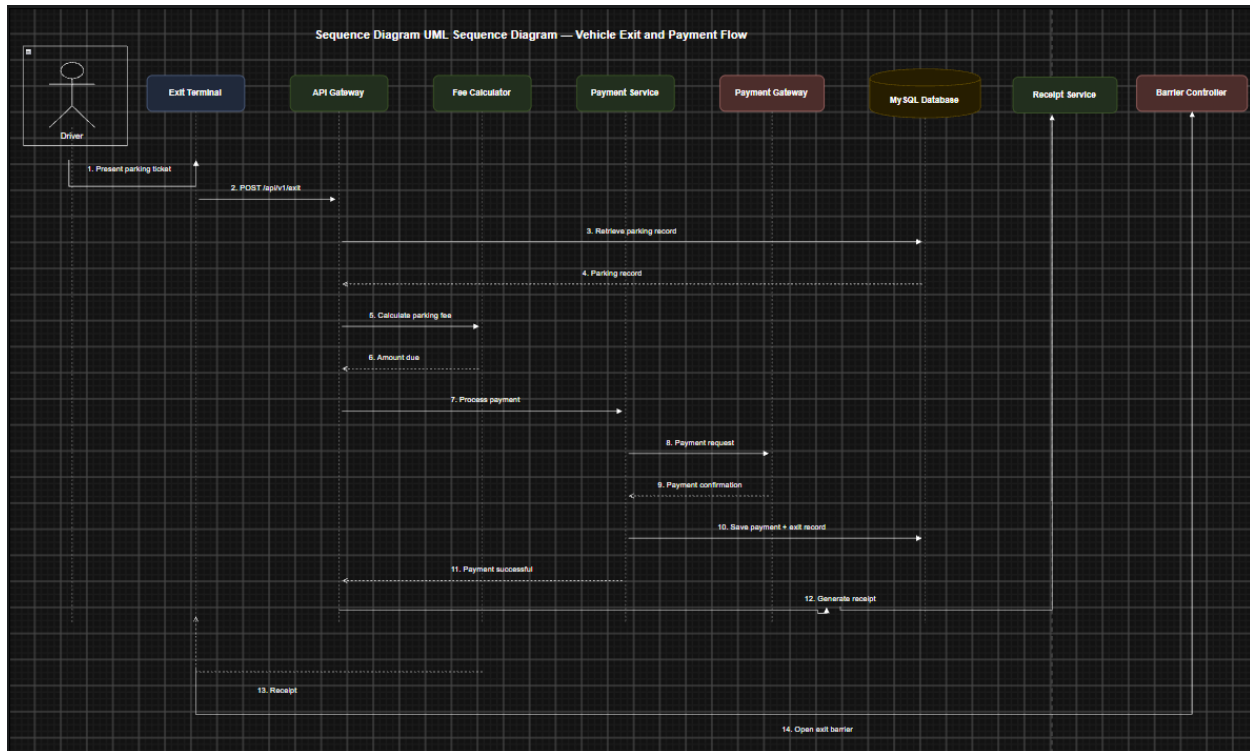
### Sequence Diagram 1 — Vehicle Entry Flow



#### Flow description:

1. Driver inserts card / presents vehicle at entry terminal.
2. Terminal captures the license plate (ALPR or manual).
3. API Gateway validates the session via Auth Service.
4. Slot Manager checks the database for an available slot and assigns it.
5. Entry record (LP, slot, timestamp, ticket ID) is saved to the database.
6. Ticket is printed/displayed and the barrier opens.

### Sequence Diagram 2 — Vehicle Exit & Payment Flow



### Flow description:

1. Driver scans ticket at exit terminal.
2. API Gateway fetches the original entry record from the database.
3. Fee Calculator computes the charge based on duration and rate schedule.
4. Fee is displayed; driver selects payment method.
5. Payment Service processes the payment via the external gateway.
6. Exit record is saved, receipt printed, and barrier opens.

## 4.3 API Design

### API 1 — Vehicle Entry

**Endpoint:** /api/v1/entry **Method:** POST **Description:** Registers a vehicle entry, assigns a slot, and creates a ticket.

#### Request Body:

```

{
  "licensePlate": "KA01AB1234",
  "entryTerminalId": "TERM-01",

```

```
    "captureMethod": "ALPR"
}
```

#### Response (200 OK):

```
{
  "ticketId": "TKT-20260906-00123",
  "slotNumber": "B-14",
  "slotType": "regular",
  "entryTimestamp": "2026-09-06T10:32:00Z",
  "status": "ENTRY_SUCCESS"
}
```

#### Error Responses:

| Code | Message               | Reason                      |
|------|-----------------------|-----------------------------|
| 409  | Parking Full          | No slots available          |
| 400  | Invalid License Plate | LP format validation failed |
| 503  | Terminal Offline      | Network/DB unreachable      |

## API 2 — Vehicle Exit & Fee Calculation

**Endpoint:** /api/v1/exit **Method:** POST **Description:** Processes vehicle exit — calculates fee, handles payment, and frees the slot.

#### Request Body:

```
{
  "ticketId": "TKT-20260906-00123",
  "exitTerminalId": "TERM-EXIT-01",
  "paymentMethod": "CARD"
}
```

#### Response (200 OK):

```
{
  "ticketId": "TKT-20260906-00123",
  "licensePlate": "KA01AB1234",
  "entryTimestamp": "2026-09-06T10:32:00Z",
  "exitTimestamp": "2026-09-06T13:15:00Z",
  "durationMinutes": 163,
  "amountCharged": 82.00,
  "currency": "INR",
  "paymentStatus": "SUCCESS",
  "receiptId": "RCP-20260906-00123",
  "status": "EXIT_SUCCESS"
}
```

## Error Responses:

| Code | Message          | Reason                                 |
|------|------------------|----------------------------------------|
| 404  | Ticket Not Found | Invalid or already used ticket ID      |
| 402  | Payment Failed   | Gateway declined or insufficient funds |
| 400  | Invalid Request  | Missing required fields                |

## 4.4 Error Handling, Logging & Monitoring

- All API errors return a consistent JSON structure: { "error": "<code>", "message": "<description>", "timestamp": "..."} }
- No sensitive data (license plates in plaintext, PANs, PINs) is written to logs.
- Structured logs (JSON format) shipped to ELK Stack for monitoring.
- Key metrics monitored: transaction failure rate, barrier open/close latency, API response times, slot occupancy percentage.
- Alerts triggered for: payment gateway downtime, > 5% transaction failure rate, system unavailability.

## 4.5 UX Design

- Entry/exit terminal UI uses large touch targets (minimum 44×44px), high-contrast colour scheme, and audio prompts for accessibility.
- Language selection available at terminal start screen.
- Admin portal is fully responsive and keyboard-navigable.
- All error messages are user-friendly (no raw error codes shown to customers).

## 4.6 Open Issues & Next Steps

| # | Issue / Enhancement                                  | Priority |
|---|------------------------------------------------------|----------|
| 1 | Integrate biometric authentication for admin access  | Medium   |
| 2 | Add QR-code based mobile entry/exit                  | Low      |
| 3 | Support multi-level parking floor management         | Medium   |
| 4 | Finalise vendor selection for ALPR camera            | High     |
| 5 | Replace ASCII diagrams with draw.io/PlantUML exports | High     |

## 5. Appendices

### 5.1 Glossary

| Term    | Meaning                                      |
|---------|----------------------------------------------|
| VPS     | Vehicle Parking System                       |
| ADR     | Architectural Decision Record                |
| ALPR    | Automatic License Plate Recognition          |
| API     | Application Programming Interface            |
| RBAC    | Role-Based Access Control                    |
| TLS     | Transport Layer Security                     |
| PCI-DSS | Payment Card Industry Data Security Standard |

### 5.2 References

- Vehicle Parking System SRS v1.0 — docs/SRS/SRS\_VehicleParkingSystem.md
- Vehicle Parking System STP v1.0 — docs/STP/STP\_VehicleParkingSystem.md
- Supplied Software Architecture and Design Specification template
- Supplied Software Requirements Specification template

### 5.3 Tools

draw.io for UML diagrams; Swagger/OpenAPI for API documentation; GitHub for version control and collaboration; Spring Boot, React.js, MySQL, Docker and ELK Stack as defined in the project SAD.